

Long-Horizon Agents

长程 Agent：如何持续推进，直到任务真正完成

长程任务为什么容易中途失效

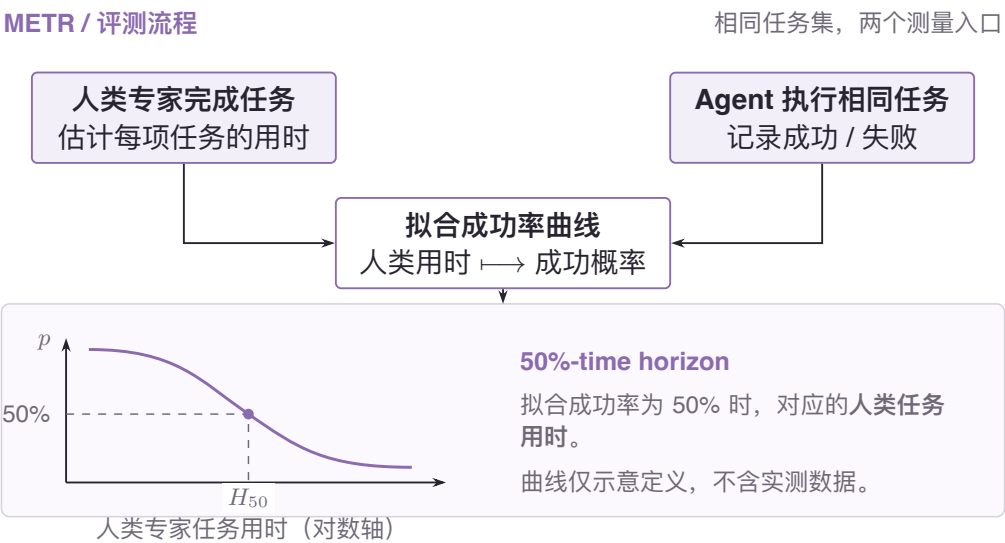
长程任务中的后续动作，往往依赖此前形成的状态。修复一个错误可能暴露另一个错误，新增功能还要检查是否破坏已有行为，因此任务路径会随执行反馈调整。

持续推进需要保留目标、当前现场、已尝试的策略和验证结果。若信息仅存在于当前对话中，上下文切换可能使它们丢失；若验收条件不清楚，也容易把部分产出当成完成。长程 Agent 因而需要可接续、可修正、可验收的过程。

METR：先明确“长”指什么

METR 的 time horizon 用人类专家完成任务的时间，作为任务长度的尺度。先估计每项任务的人类用时，再测量 Agent 的成功情况，拟合成功率随任务长度变化的曲线。

50%-time horizon 是拟合成功率为 50% 时，对应的人类任务用时。80%-time horizon 则采用更高的成功率要求。比较长程能力时，需要同时说明任务分布与可靠性门槛。



METR：任务用时与成功率共同定义 time horizon，曲线仅为示意

这个时间衡量任务难度，不能直接解释为 Agent 连续自主运行的时长，也不能推出它能处理所有相应时的人类工作。相关任务主要来自软件工程、机器学习与网络安全等领域。

这里关注的长程任务还包含多步依赖。独立小题主要增加吞吐量，连续调试复杂系统则需要保留此前尝试与现场变化。METR 提供评测方法，后面的机制支持实际推进。

ReAct：把行动接到真实反馈上

ReAct 将推理与行动交替组织：模型根据当前上下文决定动作，工具或环境执行动作，再把实际观察返回上下文。模型依据新观察调整后续计划。

ReAct / 单次交互中的行动循环



推理可以穿插于动作之间；此图省略了跨会话的存储与续跑机制。

ReAct：模型生成动作，环境返回观察，再决定下一步

整体过程可以写为：当前上下文 → 推理与动作 → 环境执行 → 新观察 → 下一步。推理可以穿插在关键动作之间，并不要求每次工具调用都先生成一段固定格式的思考。

这个循环使执行过程能够响应外部变化。跨上下文的目标保存、进度交接与恢复策略，仍需要系统另外设计。后面的 Harness 和 Goal，分别在这些位置增加约束与控制。

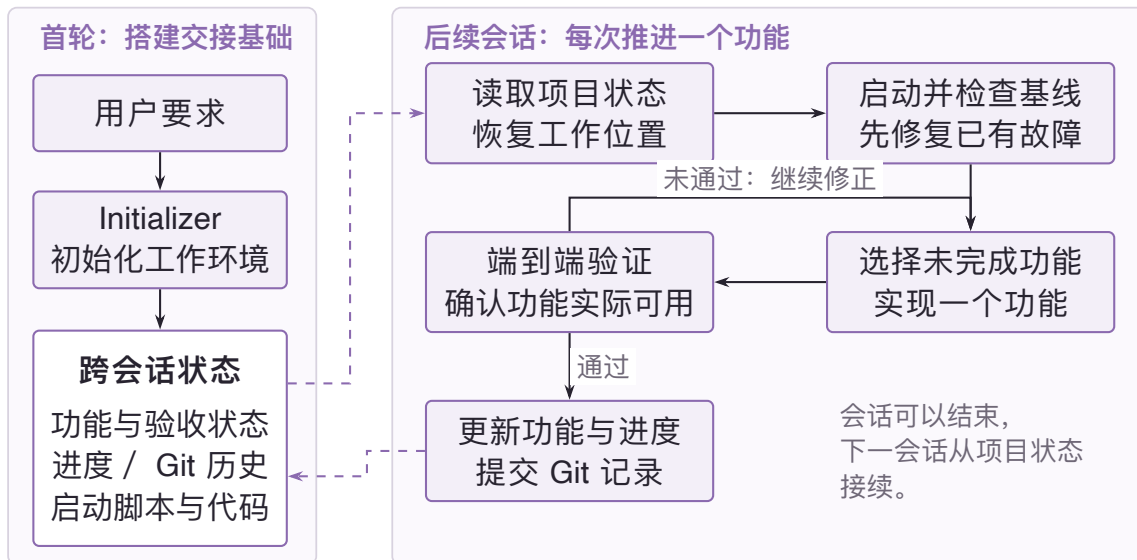
Long-running Harness：让下一次会话能够接手

Anthropic 的 long-running harness 把首轮初始化与后续增量执行分开。Initializer 先建立功能清单、启动脚本、进度文件和初始 Git 提交。后续 Coding Agent 每次读取这些记录，选择尚未完成的功能，执行、验证，再留下可接手的状态。

功能清单保存验收对象，进度文件保存已做工作与下一步线索，Git 保存代码变化和可恢复版本。会话开始时还要检查当前环境：文件记录声称完成，并不等于现场仍然可用。先启动应用、检查基础行为，再继续开发，可以尽早发现上次遗留的问题。

一次只推进有限范围，并在结束前验证和记录，能够减少半成品跨会话传递。原方案要求保护功能清单和验收要求，只在通过检查后修改 passes 状态，防止删改要求制造完成。检查本身也需要覆盖实际用户行为。

这里的 Initializer 与 Coding Agent 是不同阶段的角色，原方案通过不同的起始提示区分它们。其重点是跨会话交接协议，并不要求两套模型同时协作。该方案主要围绕 Web 应用开发验证，迁移到其他任务时，需要重新定义状态和验收方式。

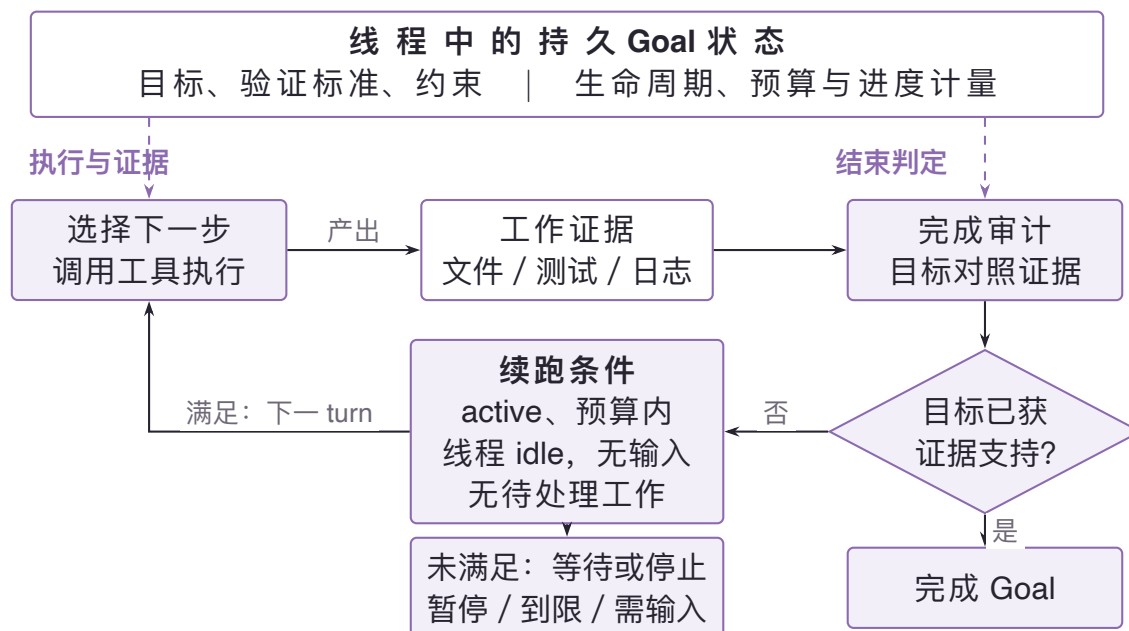


依据原方法绘制：两种角色由首轮与后续提示区分；持久文件承担交接，不表示两个 Agent 同时运行。

Long-running Harness：首轮建立外部状态，后续会话读取、推进并写回

Codex Goal：把完成条件保留在线程中

Codex Goals 将目标保存为线程中的持久状态。目标需要说明最终结果、验证依据和必须保持的约束。路径可以随新证据调整，但完成判定应持续围绕这个目标。



Codex Goal：持久目标约束多轮工作，证据决定完成，运行条件控制续跑

一轮工作结束后，系统仅当续跑条件同时满足时才继续：线程空闲，既无待处理工作也无排队的用户输入，目标仍处于活动状态，且预算允许。Agent 可以根据当前结果选择下一步，无需用户在每个中间结果后重新描述目标。

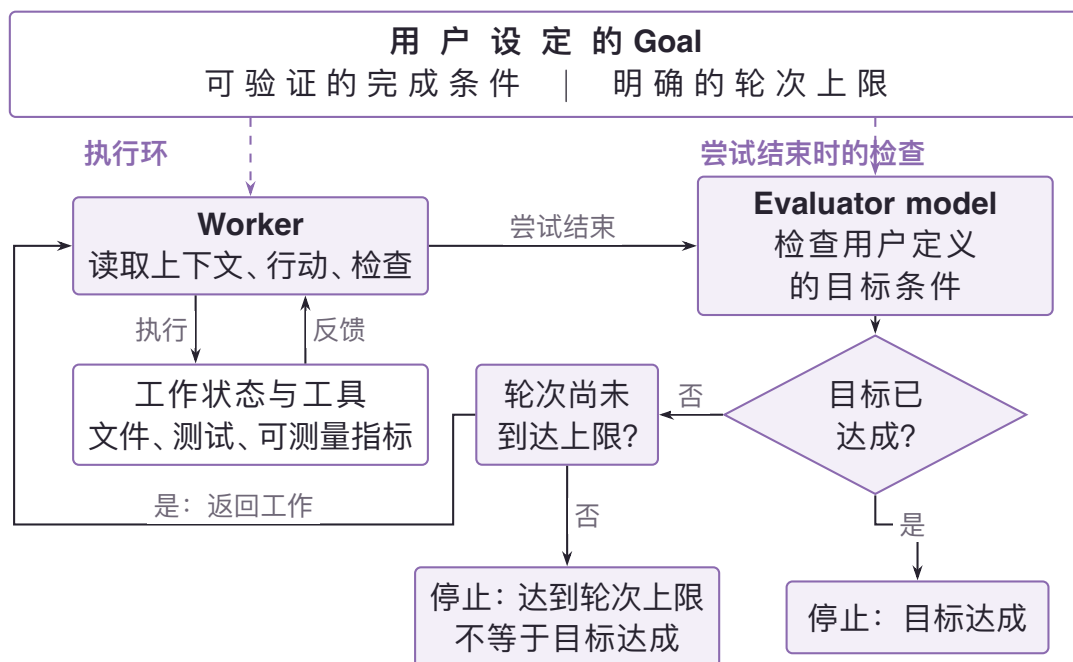
一个目标可以包含四项：结果是什么；用什么证据验收；哪些约束不能破坏；预算耗尽或需要外部信息时如何停止。

持久目标与项目文件承担不同职责。Goal 保留“为什么继续、何时完成”；文件、测试结果与日志保留现场和证据。线程中的目标也不等于全局记忆，不能据此假定其他线程自动共享全部工作状态。

达到目标、预算耗尽与遇到阻塞，含义不同。停止运行只能说明本次执行结束；只有证据支持完成条件，才可以宣告目标达成。图中展示的是公开文档描述的控制关系，没有假定存在某个未公开的独立评估模型。

Claude /goal：在尝试结束时检查目标

Claude Code 的 goal-based loop 同样要求明确的完成条件。其公开说明进一步描述了结束检查：每当执行 Agent 尝试停止，evaluator model 检查用户条件；条件尚未满足时，将任务送回继续执行，直到目标达成或达到设置的轮次上限。



Claude /goal：执行 Agent 尝试结束后，由 evaluator 检查条件并决定是否继续

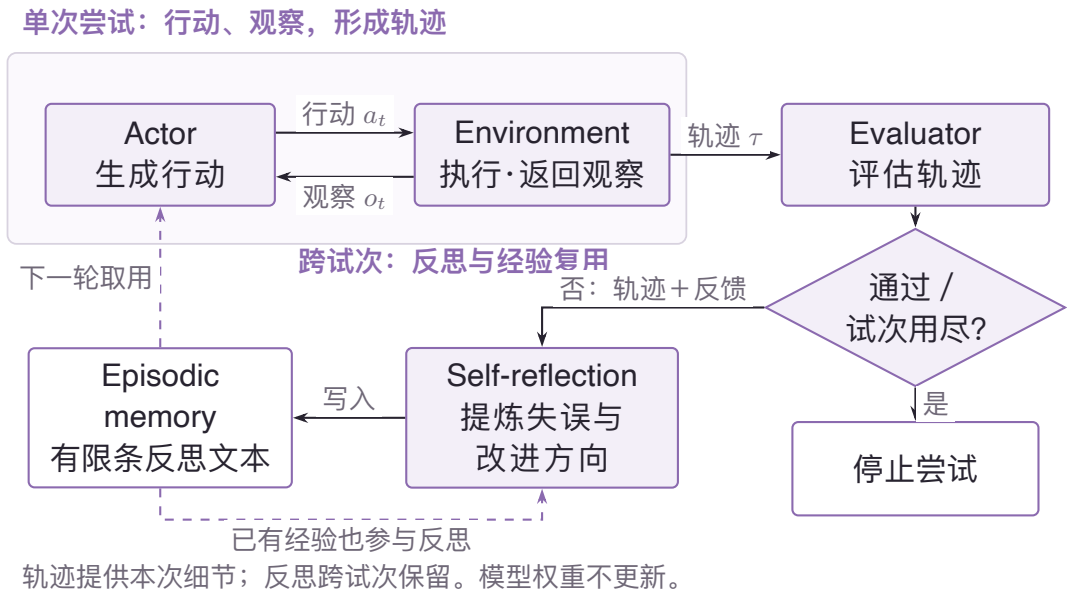
这个机制把“执行 Agent 认为可以结束”与“目标检查允许结束”分开。测试通过数量、性能分数等可测条件，能够给 evaluator 更明确的判断依据。若要求只有“做得更好”，继续与停止都会变得含糊。

evaluator 的判断仍依赖证据和评价标准。它不会自动消除遗漏、误判或测试覆盖不足。需要为重要要求提供可检查的结果，并把轮次上限视为成本边界，而非成功证明。

Goal-based loop 与定时循环的触发方式不同：前者围绕同一任务的完成条件持续迭代；/loop、/schedule 等定时入口用于到时间再次触发工作。长任务可以组合两类机制，但是否完成仍应由目标和证据决定。

Reflexion：让下一次尝试使用上一次的经验

Reflexion 在多次尝试之间增加文字形式的经验记忆。Actor 执行任务并形成轨迹，Evaluator 提供反馈；Self-reflection 结合轨迹、反馈和已有经验生成反思，再写入 episodic memory，供下一次尝试使用。



因此，重试的输入发生了变化。下一次执行既有原任务，也有“上次哪里失败、应如何调整”的经验。这个过程通过上下文中的文字反馈影响决策，不需要更新模型权重。

反思需要指向可改变的动作。仅记录“上次失败了”提供的信息有限；记录失败条件、遗漏证据和下一次应检查的位置，才更有助于修正策略。这是将该思想用于实际系统时的写入原则。

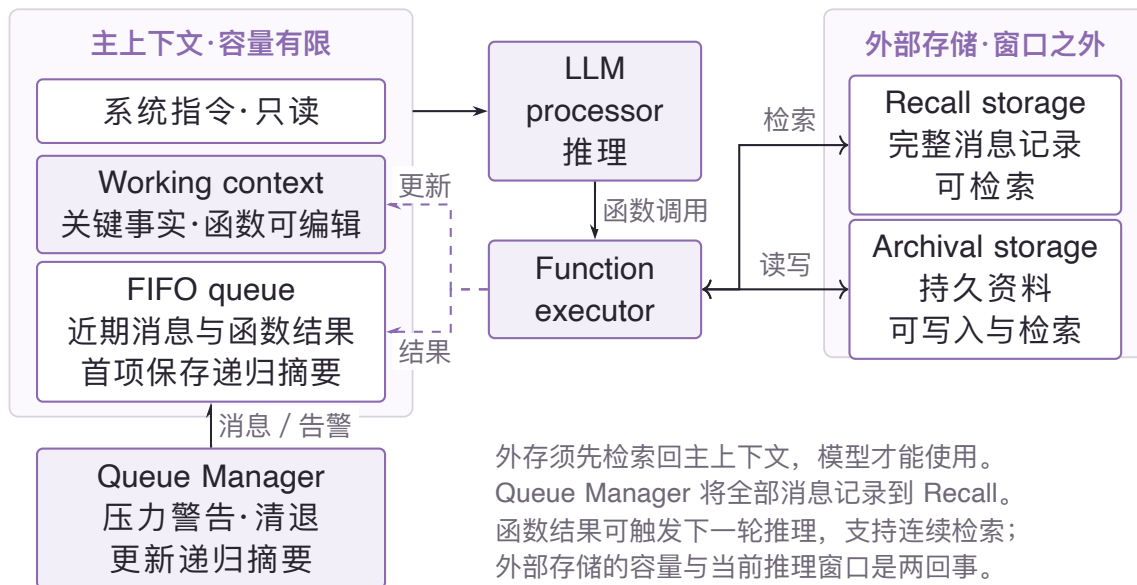
经验记忆也需要控制大小与质量。错误归因若被反复读取，可能持续误导后续尝试。Reflexion 主要验证反馈与记忆如何改善跨尝试表现，不能单凭这一机制推出跨天任务已具备可靠的恢复与验收能力。

MemGPT：让有限上下文访问外部记忆

MemGPT 借鉴操作系统的分层内存，把当前模型可见的 main context 与 external context 分开。主上下文包含系统指令、可编辑的 working context，以及保存近期交互的 FIFO 队列；外部保存可检索的历史与归档资料。

模型通过函数调用修改工作记忆、搜索历史或读取归档。外部资料被检索后，还需要作为结果进入主上下文，模型才能在当前决策中使用它。外部存储容量与当前可见信息量，是两个不同的量。

主上下文接近容量限制时，系统通过警告、摘要与队列清退管理空间。原方案会保留早期对话的递归摘要，并把历史消息保存在可检索存储中。摘要维持连续性，检索补充当前步骤所需的细节。



MemGPT: 模型在有限主上下文中工作，通过函数与队列管理访问外部记忆

对于长任务，可以据此区分稳定事实、近期工作信息与完整历史：稳定信息保留在工作记忆中，近期信息留在当前队列，更多细节按需取回。实际划分还应随任务调整。

MemGPT 的主要验证场景是长文档分析和多会话对话。它提供记忆管理机制；长任务的执行恢复、目标保持和完成检查，还需要与其他机制配合。

Voyager：把成功行为积累成能力

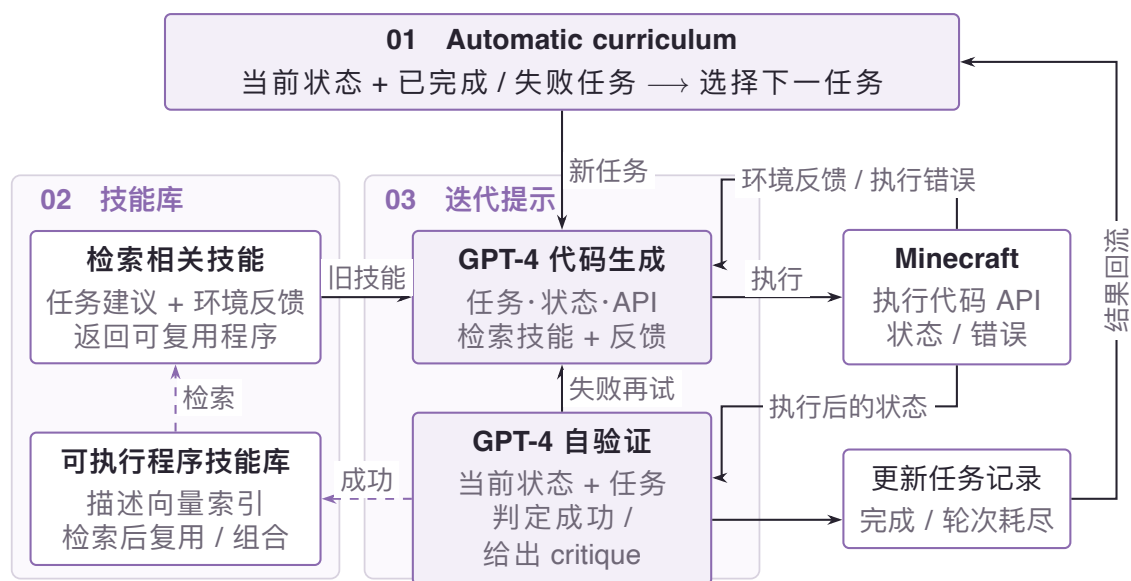
Voyager 在 Minecraft 中组织开放式探索，包含三个主要部分：Automatic Curriculum 根据当前状态与探索进展选择任务；Skill Library 保存、检索可执行代码技能；Iterative Prompting 结合已有技能和反馈生成、修正程序。

执行过程中，环境反馈、程序错误和自验证结果用于下一轮修正。成功完成的技能才进入库中，以描述的 embedding 建立索引，供后续任务检索和组合。任务结果同时反馈给课程模块，影响接下来探索什么。

这使长期运行能够在外部技能库中沉淀能力，无需微调模型权重。经验可以保存为文字反思，也可以保存为可调用代码；后者让已解决的子问题直接复用。Voyager 的 Skill 指可执行代码行为。

自验证由另一 GPT-4 critic 根据当前状态与任务检查是否完成，失败时提供修正线索。原方案每项任务最多进行四轮代码生成，仍失败则转向其他任务，避免无止境地重试。

这些结果来自特定的游戏环境，代码技能也受环境 API 约束。其值得借鉴的部分，是任务选择、反馈修正与成功能力积累之间的连接方式；迁移到真实工程仍需重新设计验证与技能边界。



积累的是可组合的程序能力；通过提示与外部技能库学习，无需微调模型权重。

Voyager：课程选择任务，技能支持执行，反馈修正代码，成功行为进入技能库

这些机制分别保存了什么

目标约束、现场事实、经验和技能承担不同职责，可以分别保存，再在当前步骤组合使用。各项工作对应的作用位置如下。

机制	主要对象	作用位置
METR	任务长度与成功率	衡量系统能处理多难的任务
ReAct	当前观察与行动轨迹	依据环境反馈决定下一步
Long-running Harness	功能清单、进度与代码状态	让后续会话接续工作
Codex / Claude Goal	目标、完成条件与运行边界	约束持续执行和退出
Reflexion	文字形式的经验	改进下一次尝试的策略
MemGPT	工作记忆、历史与归档	按需恢复当前决策所需信息
Voyager	可执行代码技能	复用成功行为并支持后续探索

一个工程系统可以保存持久目标，用增量会话推进项目，把反馈转成经验，再通过外部记忆取回细节。技能库则适合保存值得反复调用的稳定行为。

这种组合是设计上的归纳。各项工作分别验证了自己的机制与场景，组合后的整体可靠性仍需重新评测。

怎样确认持续运行带来了进展

首先检查目标。新的发现可以改变计划，但不能悄悄缩小验收范围。完成记录应对应最初要求，并给出验证证据。

再检查状态。恢复任务时，核对文件版本、环境和实际产物。记录描述过去的观察，重要事实仍需现场确认；有冲突时先处理冲突。

最后检查进展。新增产物、验证结果、可复用经验或已确认的外部等待，都能支持后续决策。若多轮重复计划或同一失败，则需调整方法、补充信息或明确停止原因。

评测应记录完成率、成本、人工介入与恢复表现，并采用可比的任务集、工具条件与预算。运行更久本身无法证明能力提高。

总结

- (1) 长程能力需要结合任务难度与可靠性理解。METR 提供一种衡量方式；运行时长本身不能证明任务完成能力。
- (2) 持续推进需要目标、现场与经验能够接续。ReAct 连接行动与反馈，Harness 管理会话交接，Goal 明确完成条件和续跑控制，Reflexion 与 MemGPT 分别支持经验使用和记忆调度。
- (3) 长期工作还可以积累可复用能力，Voyager 展示了代码技能库的作用。无论采用哪种组合，最终都要回到同一个检查：原始目标是否达成，当前证据是否足以支持这一结论。